Andrew Long
Prof Jon McGown
CSIS-2430-501 - Summer
DATE

Programming Project 1

For our first programming assignment of the semester we took a look and did some experimentation with sorting algorithms. A sorting algorithm is an algorithm that takes a collection of items and orders them in a specified sequence. These algorithms are important as they are used heavily in the field of computer science. Online stores use these to sort their catalogs, and their customers can even customize these to tailor for items they are looking for. Taking a deeper look into these algorithms and comparing the results is a good way to understand how algorithmic efficiency affects large scale systems and datasets. As we will go over in the future parts of this paper, you can see that some of these algorithms work much faster and efficiently than some of the others, and as the data set they are sorting grows that speed and efficiency become all the more important.

My program works by creating multiple arrays of integers using java's random function, the arrays are created using specified sizes which are found in my sizes *array* in my *main* method. Each randomly generated array is then passed into four separate sorting algorithms. Comparisons made by each algorithm are counted, and when all the arrays have been sorted, the program displays the sorted arrays along with the number of comparisons required to place the elements in the correct order. To ensure the accuracy of the sorting algorithms, I created two JUnit test cases for each sorting algorithm, one with a small number of integers and one with a larger number of elements. To check for edge cases, I included multiples of some elements within the larger element testing blocks.

For merge sort, the program begins by splitting an array into two separate arrays down the middle. This process then continues on recursively, dividing each half into more halves until all the elements of the original array have been isolated. Once we have reached the point where all of our array has been broken down into individual elements, a separate helper method is called which we use to recombine the arrays. The way we merge these elements is by comparing the first unmerged element of the left and right halves and placing the smaller of the two values into the output array. It is at this comparison that we increment a counter for a variable *mergeCount* for every comparison made. This continues until all elements from both halves have been merged back into a single sorted array. During each comparison we are incrementing a counter to keep track of these comparisons. Once the program compares the base elements and combines them into groups of two, it loops through and compares the elements left to right with the elements of another group left to right. This continues until we are left with a single sorted array. For me, this was the easiest of the algorithms to implement as it made the most logical sense to myself to work through, a tutorial I read for this online referred to

this as the divide and conquer method and the thought behind this process and its given nickname just clicked.

The quick sort algorithm calls for a pivot. In my implementation, I used the last item in the array as the pivot point. Once we have a pivot we create two pointers, a left one that starts at index 0 and moves forward in the array, and a right one that starts at the end and moves backwards. The pointers move toward each other comparing elements of the array along the way, each of these comparisons are being counted and stored in the *quickComparisons* variable. When both pointers stop on values that are on the wrong side of the pivot, those elements are swapped. This continues until the pointer meets one another, at which point we move our pivot into the correct position in the array and the array is split in two. This is where the recursion for this algorithm takes place as another pivot is set and more comparisons are made, elements swapped, until we work through and our whole array is sorted.

The heap sort for me was the one that proved most challenging. Even with this sorting algorithm in working order, parts of the logic behind it are still a bit cloudy to myself. I had not worked with heaps, or trees before so this was a lot of new vocabulary for me and logic I was not familiar with. I do see that we have some things in module 5 regarding trees so hopefully that section will help in my understanding of this algorithm. What the algorithm does is take our array, and transform it into a heap. Once the heap is created, the *heapify* method compares the parent nodes with its left and right children, we keep track of these comparisons at this step and increment our *heapComparisons* variable. If one of the children elements happens to be larger than the parent element, the values are swapped. The program uses recursion to loop through and works through these comparisons and swaps until everything in our heap has been extracted and placed back into a sorted array.

The shaker sort algorithm works by repeatedly moving in both directions of the array to move elements towards their correct positions. It begins by performing a left to right pass through the array, comparing adjacent elements along the way and swapping them whenever they are out of order. Once we reach the end of the array the program begins to move right to left doing the same thing but in the opposite direction. Each of these comparisons is tracked by our *shakerComparisons* variable. These passes continue until the array is able to be passed through without any swaps, indicating the array is now sorted. This could have been a contender for the easiest algorithm to implement, but most of my time spent on this sorting algorithm was actually held captive by working on a different sorting algorithm with a similar name, muddying my first experience with the shaker sorting algorithm.

When it comes to counting the comparisons made in each algorithm, each algorithm, as noted in the paragraphs above, had a variable that was being incremented during each element comparison of the program. With my heap sort, I had some concerns with where to place the incrementor, and with how the logic behind this method is just not making sense to myself like the others, creating arrays on paper and trying to compare my comparisons to the program or writing jUnit test cases for this specific algorithm proved difficult. That said, even if the comparison count is not perfectly accurate, it is consistent. It is consistent. Without consistency

in our comparison counts, making any informed opinions on the difference of the efficiency between algorithms would not be possible. Any insight we would hope to gain from these values would ultimately be moot, as they would have been informed from incorrect or false data.

Based on the runs of the program I have conducted, the merge sort algorithm is clearly the most effective. For me, this did not seem intuitive, as the algorithm essentially forces all the elements to be compared, where I thought maybe the fact that some things, even in randomly generated arrays would show up already in order and that this extra division of elements would be at times unnecessary. The effectiveness of these comparisons becomes more apparent as the arrays they are sorting through become larger. As the data sets become larger, we can remove the likelihood of chance I was referring to earlier, with a data set of 4 we might only need to do a few comparisons, and they all might be right first pass. Where a data set of 100 and how that is going to be sorted will rely much heavier on the effectiveness of the sorting algorithm itself rather than the elements in the array, by luck or by chance, lining up perfectly to be sorted by a specific sorting style. This behavior helps explain why smaller input sizes do not always reflect the theoretical performance predicted by Big-O analysis. I will share the results of one of these runs in a table below.

Table 1. Comparison Counts for Each Sorting Algorithm

Array Size n	Merge	Quick	Heap	Shaker
4	5	6	6	6
6	9	12	16	12
8	16	23	25	27
44	189	309	336	775
66	314	470	608	1550
88	457	805	847	2967

If I wanted to continue this experiment, and further test these algorithms I would want to increase my data sets, and vary my data types. How would these results differ when sorting things alphabetically rather than numerically? What would these look like in the scale of the thousands, with thousands of runs tested and averages being accumulated? These would all be questions I would want to answer in my continued experimentation on this subject matter. As for improved performance through slight change of implementation, in some of the readings I did on the quick sort the placement of the pivot is a point of question. Some people said placing the pivot at the end, as I did, was a good way to structure the algorithm, while others said that

placing the pivot randomly may actually increase the effectiveness of the sorting and reduce the number of comparisons.

In conclusion, this project provided some valuable insight into the effectiveness of sorting algorithms, and their efficiency can scale with larger projects or data sets. The comparison data collected through our counters show that in small scale projects these differences in efficiency are rather unimportant, but for the engineers and data scientists working on large scale projects implementing the right algorithm can save exponentially on computing power. Overall, this project helped highlight the significance in software efficiency and the importance of this coursework with practical real world applications.

Sources

Sambol, M. (n.d.). *Michael Sambol [YouTube channel]*. YouTube. https://www.youtube.com/@MichaelSambol. Accessed June 12, 2026.

YouTube. (n.d.). *Sorting algorithm visualization / explanation video* [Video]. YouTube. https://www.youtube.com/watch?v=4VqmGXwpLqc. Accessed June 12, 2026.

YouTube. (n.d.). *Quick sort / partitioning explanation video* [Video]. YouTube. https://www.youtube.com/watch?v=Hoixgm4-P4M&t=103s. Accessed June 12, 2026.

YouTube. (n.d.). *Sorting algorithm tutorial (comparison-based sorting)* [Video]. YouTube. https://www.youtube.com/watch?v=2DmK_H7IdTo. Accessed June 12, 2026.

YouTube. (n.d.). *Sorting algorithm visualization (short)* [Short video]. YouTube. https://www.youtube.com/shorts/0X1cmNdLp2E. Accessed June 12, 2026.

YouTube. (n.d.). *Heap sort / sorting algorithm explanation* [Video]. YouTube. https://www.youtube.com/watch?v=h8eyY7dIiN4&t=1199s. Accessed June 12, 2026.

YouTube. (n.d.). *Sorting algorithm implementation walkthrough* [Video]. YouTube. https://www.youtube.com/watch?v=bOk35XmHPKs&t=1105s. Accessed June 12, 2026.